

Cursor Feature Development Guide

Q&A Content Management System with Dual-Prompt Integration

Document Overview

This guide instructs Cursor to build a comprehensive GUI feature for content professionals to manage, edit, and migrate Q&A content generated by dual prompts (Question Generator + Answer Generator). The system integrates with your existing database and provides end-to-end content lifecycle management.

1. FEATURE SPECIFICATION & REQUIREMENTS

1.1 Feature Name

Q&A Content Studio - A unified GUI for managing AI-generated question and answer pairs with corpus

validation, tone management, and batch migration capabilities.

1.2 User Persona

- **Content Professionals:** Non-technical staff managing Army recruitment Q&A content
- **Subject Matter Experts (SMEs):** Reviewing and validating answer accuracy
- **Content Managers:** Publishing, archiving, and migrating content

1.3 Core Objectives

- Load and manage corpus files (like goarmysite.txt)
- Execute Question Generator prompt via API/UI button
- Display generated questions in editable table format
- Execute Answer Generator prompt on question batches
- Display 4 answer variants with full metadata
- In-app editing of questions, answers, tone, links, and keywords
- Real-time validation against corpus file
- Batch approval/rejection workflow
- Migrate validated Q&A pairs to production database
- Version history and audit trail
- Export to multiple formats (CSV, JSON, Markdown)

1.4 Success Criteria

Criterion	Target	Measurement
-----------	--------	-------------

Time to generate 250 Q&A pairs	<60 minutes	End-to-end workflow timing
Zero manual reformatting needed	100%	Direct prompt output → system
Content editing time saved	40% reduction	Before/after editing time
System uptime	99.5%	Availability monitoring
User satisfaction	4.5/5.0	User feedback survey

2. DATABASE SCHEMA DESIGN

2.1 Core Tables to Create/Modify

Table: qa_projects

```
CREATE TABLE qa_projects (  
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),  
  name VARCHAR(255) NOT NULL,  
  description TEXT,  
  status ENUM('draft', 'in_progress', 'review', 'published', 'archived') DEFAULT 'draft',  
  corpus_file_id UUID REFERENCES corpus_files(id),  
  created_by UUID REFERENCES users(id),  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  question_generator_prompt_id UUID,  
  answer_generator_prompt_id UUID,  
  target_question_count INT DEFAULT 250,  
  published_count INT DEFAULT 0,  
  archived_at TIMESTAMP NULL  
);
```

Table: questions

```
CREATE TABLE questions (  
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),  
  project_id UUID NOT NULL REFERENCES qa_projects(id),
```

```

    topic VARCHAR(255) NOT NULL,
    persona VARCHAR(100),
    tone VARCHAR(100),
    question_text TEXT NOT NULL,
    source_urls TEXT ARRAY,
    status ENUM('draft', 'approved', 'rejected', 'published') DEFAULT 'draft',
    batch_number INT,
    created_from_generator BOOLEAN DEFAULT true,
    edited_by UUID REFERENCES users(id),
    edited_at TIMESTAMP NULL,
    corpus_validated BOOLEAN DEFAULT false,
    validation_notes TEXT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

```

Table: answers

```

CREATE TABLE answers (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    question_id UUID NOT NULL REFERENCES questions(id) ON DELETE CASCADE,
    variant_level ENUM('very_direct', 'direct', 'somewhat_direct', 'indirect') NOT NULL,
    answer_text TEXT NOT NULL,
    character_count INT,
    keywords TEXT ARRAY,
    source_link VARCHAR(500),
    qa_validation_status ENUM('valid', 'needs_verification', 'invalid', 'pending') DEFAULT 'valid',
    validation_notes TEXT,
    edited_by UUID REFERENCES users(id),
    edited_at TIMESTAMP NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

```

Table: corpus_files

```

CREATE TABLE corpus_files (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    name VARCHAR(255) NOT NULL,
    file_content TEXT NOT NULL,
    file_hash VARCHAR(64),
    source_urls TEXT ARRAY,
    created_by UUID REFERENCES users(id),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    is_active BOOLEAN DEFAULT true
);

```

Table: batch_processing_jobs

```
CREATE TABLE batch_processing_jobs (  
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),  
  project_id UUID NOT NULL REFERENCES qa_projects(id),  
  batch_number INT NOT NULL,  
  questions_in_batch INT,  
  status ENUM('pending', 'processing', 'completed', 'failed') DEFAULT 'pending',  
  prompt_type ENUM('question_generator', 'answer_generator') NOT NULL,  
  api_response JSONB,  
  error_message TEXT,  
  started_at TIMESTAMP NULL,  
  completed_at TIMESTAMP NULL,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

Table: content_audit_log

```
CREATE TABLE content_audit_log (  
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),  
  project_id UUID REFERENCES qa_projects(id),  
  question_id UUID REFERENCES questions(id),  
  answer_id UUID REFERENCES answers(id),  
  action_type VARCHAR(100),  
  old_value JSONB,  
  new_value JSONB,  
  changed_by UUID REFERENCES users(id),  
  changed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

2.2 Database Indexes (Performance)

```
CREATE INDEX idx_questions_project ON questions(project_id);  
CREATE INDEX idx_questions_status ON questions(status);  
CREATE INDEX idx_answers_question ON answers(question_id);  
CREATE INDEX idx_answers_validation ON answers(qa_validation_status);  
CREATE INDEX idx_batch_project ON batch_processing_jobs(project_id);  
CREATE INDEX idx_corpus_active ON corpus_files(is_active);  
CREATE INDEX idx_audit_project ON content_audit_log(project_id);  
CREATE INDEX idx_audit_timestamp ON content_audit_log(changed_at);
```

3. ARCHITECTURE & SYSTEM DESIGN

3.1 High-Level Data Flow

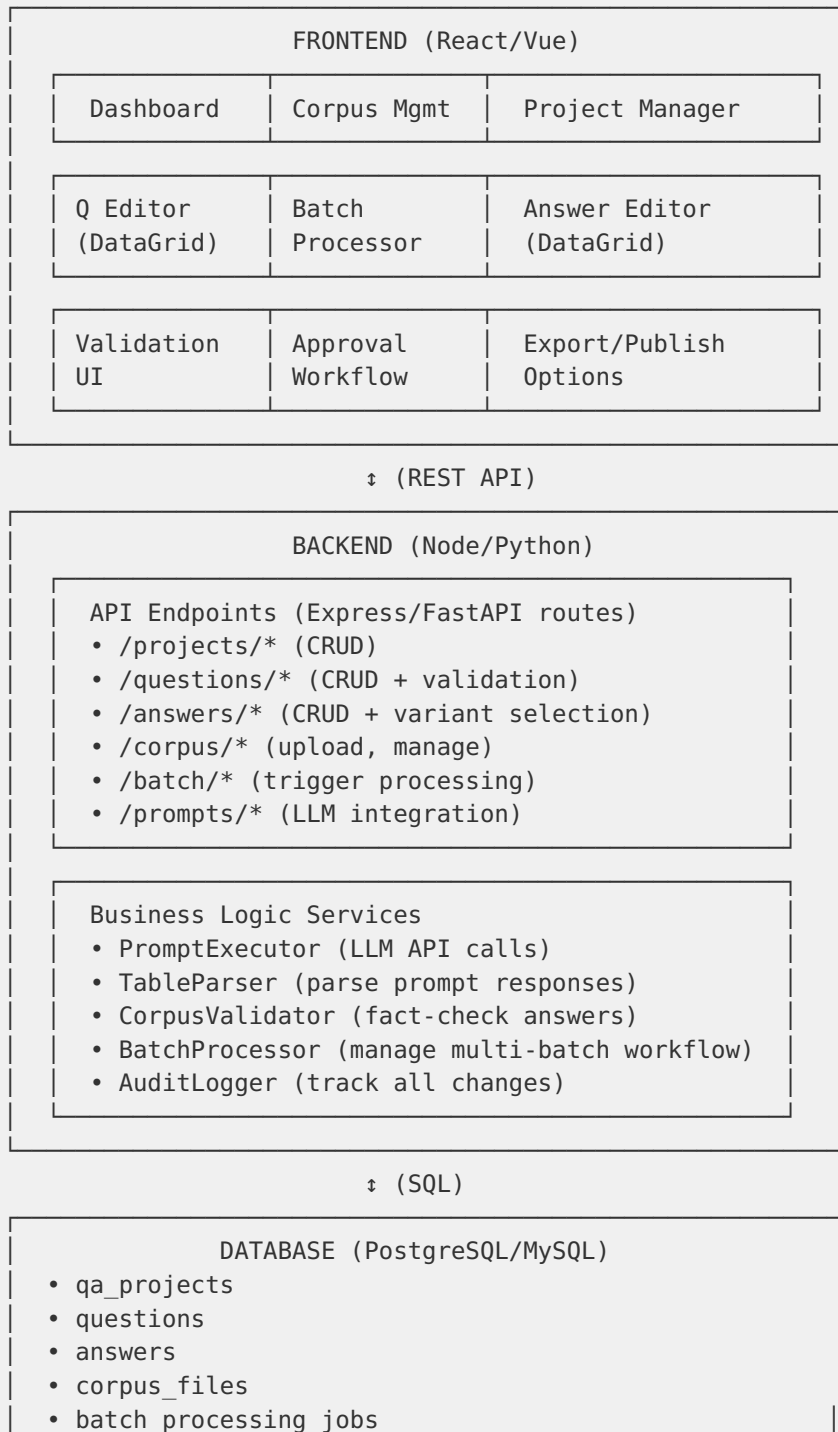
```
USER UPLOADS CORPUS FILE
↓
USER ENTERS PROJECT DETAILS
↓
SYSTEM INITIALIZES PROJECT + SAVES CORPUS
↓
USER CLICKS "GENERATE QUESTIONS"
↓
SYSTEM CALLS QUESTION GENERATOR PROMPT (via API)
↓
PROMPT RETURNS 250 QUESTIONS IN TABLE FORMAT
↓
SYSTEM PARSES TABLE → STORES IN questions TABLE
↓
SYSTEM DISPLAYS QUESTIONS IN EDITABLE GRID UI
↓
USER REVIEWS/EDITS QUESTIONS (optional)
↓
USER SELECTS 25 QUESTIONS → CLICKS "GENERATE ANSWERS"
↓
SYSTEM CALLS ANSWER GENERATOR PROMPT (Batch 1 of 10)
↓
PROMPT RETURNS 4 ANSWER VARIANTS PER QUESTION
↓
SYSTEM PARSES RESPONSE → STORES IN answers TABLE
↓
SYSTEM DISPLAYS ANSWERS IN EDITABLE GRID UI
↓
USER REVIEWS/SELECTS BEST VARIANTS
↓
USER CLICKS "VALIDATE AGAINST CORPUS"
↓
SYSTEM RUNS CORPUS VALIDATION (fuzzy matching)
↓
SYSTEM FLAGS ANSWERS NEEDING VERIFICATION
↓
USER REVIEWS FLAGS → APPROVES/REJECTS/EDITS
↓
REPEAT FOR REMAINING 9 BATCHES
↓
USER APPROVES ALL Q&A PAIRS
↓
USER CLICKS "PUBLISH TO DATABASE"
↓
SYSTEM MIGRATES ALL Q&A TO PRODUCTION db
↓
```

SYSTEM CREATES AUDIT LOG ENTRIES

↓

PROJECT STATUS CHANGES TO "PUBLISHED"

3.2 Component Architecture



- content_audit_log

4. FRONTEND UI/UX REQUIREMENTS

4.1 Main Dashboard Screen

Q&A Content Studio Dashboard

[Menu ≡]

New Project

Import CSV

Settings

Active Projects

Project Name	Status	Q Count	A Count	Action
Army 2024 Q1	In Prog	250	125	[Edit]
Careers Q2	Draft	0	0	[Start]
Benefits 2023	Published	300	300	[View]

Recent Activity

- 3 hours ago: 50 answers generated in Army 2024 Q1
- 1 day ago: Corpus file updated (goarmysite.txt)
- 2 days ago: 100 answers published to database

4.2 Project Setup Wizard

Step 1: Project Info

Create New Project

Project Name:

Description:

Target Q Count:

[250]

[Next]

[Cancel]

Step 2: Upload Corpus File

Upload Corpus File

[Drag & Drop or Click to Upload goarmysite.txt]
File size: 85 KB | Format: .txt
MD5: 3f8c2a...
[Back] [Next] [Cancel]

Step 3: Configure Prompts

Select Prompts to Use

Question Generator Prompt:
[Dropdown: "Default Q-Gen v2.1"]

Answer Generator Prompt:
[Dropdown: "Default A-Gen v2.1"]

☒ Use corpus validation for answers
☒ Auto-extract keywords from answers
☒ Create audit log

[Back] [Create Project] [Cancel]

4.3 Question Editor Grid

Army 2024 Q1 - Questions					[Batches: 1]	
[Generate Questions] [Edit Selected] [Delete] [Export]						
☒	Topic	Persona	Tone	Question Text	Status	Edit
☒	Career Prospect in Army	Nutr	Where to start with military career paths?	Draft		
☒	Benefits & Pay	Vet	Direc t	How much do soldiers earn	Approve 	
☒	Joining Process	Parent Indi rect	What is the ASVAB?	Draft		
[Load More] Showing 1-3 of 250						

CLICK [🔍] → Opens Edit Modal:

Edit Question

[X]

Topic: [Career in Army_____]
Persona: [Prospect_____]
Tone: [Neutral_____]
Question: [Where to start with...][Character: 45]
Source URLs: [✓ url1 ✓ url2 □ url3]
Status: [Draft ▼]
Notes: [_____]
[Save] [Cancel]

4.4 Answer Variant Selector

Answer Variants for: "Where to start with military..."
Status: [Needs Verification] [Validate vs Corpus]

SELECT YOUR PREFERRED VARIANT:

- ☐ VARIANT 1: Very Direct (Recommended)
"Start by contacting a recruiter. They'll explain all options and help you find the right career..."
[Length: 142] [Keywords: recruiter, career, options]
[Source: url1] [Edit] [Mark as Invalid]
- ☐ VARIANT 2: Direct
"The first step is to meet with an Army recruiter who can discuss your background, interests and the available military occupational specialties..."
[Length: 158] [Keywords: recruiter, background, MOS]
[Source: url1] [Edit] [Mark as Invalid]
- ☐ VARIANT 3: Somewhat Direct
"Most people interested in military service begin by learning about different career paths. The Army offers over 150 specializations..."
[Length: 167] [Keywords: military, career, Army]
[Source: url2] [Edit] [Mark as Invalid]
- ☐ VARIANT 4: Indirect
"Military careers are quite diverse. Before making any decisions, it's helpful to understand the many different roles available and what they entail..."
[Length: 172] [Keywords: careers, military, roles]
[Source: url1, url2] [Edit] [Mark as Invalid]

[Save Selection] [Edit All Variants] [Skip] [Next ►]

4.5 Batch Processing View

Batch Processing - Army 2024 Q1

Overall Progress: 50%
Batch 1-3 Complete | Batch 4 Processing | 6 Pending

Batch #	Status	Progress	Q Count	Time	Action
1	✓Complete		25/25	2m 15s	[View]
2	✓Complete		25/25	2m 08s	[View]
3	✓Complete		25/25	2m 22s	[View]
4	⌚Processing		12/25	1m 45s	[Cancel]
5	Pending		0/25	-	[Start]
6	Pending		0/25	-	[Start]
... (10 total batches)					

[Start All]

[Pause All]

[Cancel All]

[Continue Later]

[Export Progress Report]

4.6 Validation & Publishing Workflow

Approval & Publishing Workflow

Project Status: [In Progress]
Questions Ready: 250/250 ✓
Answers Complete: 250/250 ✓
All Validated: 248/250 ⚠

Issues Requiring Review:
⚠ 2 answers flagged "Needs Verification"
[Q15: "What is the ASVAB?" Answer variant 2]
[Q247: "How long is..." Answer variant 4]
[View Details]

[Validate All]

[Approve All]

[Force Publish]

Publishing Options:

Publish to main database

Create backup before publishing

Notify team via email

Generate audit report

[Publish Now]

[Schedule]

[Cancel]

After Publishing:

✓ Publishing Complete

- 250 questions published
- 1,000 answer variants stored (4 per question)
- 2,847 keywords extracted and indexed
- Audit log created with 250 entries
- Backup saved: backup_20240115_143022.sql
- Email notification sent to team

Published Content Available At:

<https://api.goarmy.com/qa/published/2024-q1>

[\[View Published\]](#) [\[Export Report\]](#) [\[Back to Dashboard\]](#)

5. BACKEND API ENDPOINTS

5.1 Projects Endpoints

POST /api/projects

Create new Q&A project

Request:

```
{
  "name": "Army 2024 Q1",
  "description": "Q&A for spring recruitment",
  "target_question_count": 250,
  "corpus_file_id": "uuid-of-corpus"
}
```

Response: { "id": "project-uuid", "status": "draft", ... }

GET /api/projects

List all projects

Query params: ?status=in_progress&limit=20&offset=0

Response: { "data": [...], "total": 5, "offset": 0 }

GET /api/projects/:projectId

Get project details with counts

```
Response: {
  "id": "uuid",
  "name": "Army 2024 Q1",
  "status": "in_progress",
  "question_count": 250,
```

```

    "answer_count": 1000,
    "approved_count": 998,
    "corpus_file": { ... }
  }

PUT /api/projects/:projectId
Update project
Request: { "status": "review", "description": "..." }
Response: { "id": "uuid", "updated_at": "2024-01-15..." }

DELETE /api/projects/:projectId
Archive/delete project
Response: { "success": true, "archived_at": "2024-01-15..." }

```

5.2 Questions Endpoints

```

POST /api/projects/:projectId/questions
Create single question or batch import
Request (Single):
{
  "topic": "Careers",
  "persona": "Prospect",
  "tone": "Neutral",
  "question_text": "What are the career options?",
  "source_urls": ["url1", "url2"]
}

Request (Batch - from prompt):
{
  "batch_data": { markdown_table },
  "batch_number": 1
}

Response: { "created_count": 25, "questions": [...] }

GET /api/projects/:projectId/questions
List all questions in project
Query: ?status=draft&batch=1&sort=created_at
Response: { "data": [...], "total": 250 }

GET /api/projects/:projectId/questions/:questionId
Get single question with all answers
Response: {
  "id": "q-uuid",
  "question_text": "...",
  "answers": [
    { "variant_level": "very_direct", "text": "...", ... },
    ...
  ]
}

```

PUT /api/projects/:projectId/questions/:questionId

Edit question

```
Request: {
  "question_text": "updated text",
  "tone": "direct",
  "status": "approved"
}
```

```
Response: { "id": "q-uuid", "updated_at": "..." }
```

DELETE /api/projects/:projectId/questions/:questionId

Delete question (and cascade answers)

```
Response: { "success": true }
```

5.3 Answers Endpoints

POST /api/projects/:projectId/questions/:questionId/answers

Create answer variants

```
Request: {
  "variants": [
    { "level": "very_direct", "text": "...", "keywords": [...] },
    { "level": "direct", "text": "...", "keywords": [...] }
  ],
  "source_link": "url1"
}
```

```
Response: { "created_count": 4, "answers": [...] }
```

GET /api/projects/:projectId/questions/:questionId/answers

Get all variants for question

```
Response: {
  "question_id": "q-uuid",
  "variants": [
    { "id": "a1", "level": "very_direct", "text": "..." },
    ...
  ]
}
```

PUT /api/projects/:projectId/answers/:answerId

Update answer variant

```
Request: {
  "text": "updated answer",
  "keywords": ["new", "keywords"],
  "qa_validation_status": "valid"
}
```

```
Response: { "id": "a-uuid", "updated_at": "..." }
```

PUT /api/projects/:projectId/questions/:questionId/answers/select

Select best variant for question

```
Request: { "selected_answer_id": "a-uuid" }
```

```
Response: { "success": true }
```

5.4 Prompt Execution Endpoints

POST /api/prompts/question-generator/execute

Execute question generator prompt

```
Request: {
  "urls": ["url1", "url2", ...],
  "target_count": 250,
  "prompt_version": "2.1"
}
```

```
Response: {
  "execution_id": "exec-uuid",
  "status": "processing",
  "estimated_time": 45000 # ms
}
```

GET /api/prompts/executions/:executionId

Check execution status

```
Response: {
  "status": "completed",
  "result": { markdown_table },
  "tokens_used": 2847,
  "completed_at": "2024-01-15..."
}
```

POST /api/prompts/answer-generator/execute

Execute answer generator on batch

```
Request: {
  "project_id": "uuid",
  "batch_number": 1,
  "questions": [ { id, text, ... }, ... ],
  "corpus_file_id": "uuid"
}
```

```
Response: {
  "execution_id": "exec-uuid",
  "status": "processing",
  "batch_number": 1,
  "questions_count": 25
}
```

5.5 Corpus Management Endpoints

POST /api/corpus/upload

Upload new corpus file

Request: FormData (multipart/form-data)

- file:
- name: "goarmysite.txt"

```
Response: {
  "id": "corpus-uuid",
  "name": "goarmysite.txt",
}
```

```

    "file_size": 87520,
    "file_hash": "3f8c2a...",
    "source_urls": ["url1", "url2", ...]
  }

GET /api/corpus
List all corpus files
Response: { "data": [...], "total": 5 }

GET /api/corpus/:corpusId
Get corpus file content
Response: {
  "id": "corpus-uuid",
  "content": "full file content...",
  "file_size": 87520
}

PUT /api/corpus/:corpusId
Update corpus file
Response: { "id": "corpus-uuid", "updated_at": "..." }

DELETE /api/corpus/:corpusId
Delete corpus file
Response: { "success": true }

```

5.6 Validation Endpoints

```

POST /api/validation/corpus-check
Validate answers against corpus
Request: {
  "answers": [
    { "id": "a1", "text": "...", "keywords": [...] },
    { "id": "a2", "text": "...", "keywords": [...] }
  ],
  "corpus_file_id": "corpus-uuid"
}
Response: {
  "results": [
    { "answer_id": "a1", "status": "valid", "confidence": 0.95 },
    { "answer_id": "a2", "status": "needs_verification",
      "flags": ["No direct mention of X", "Requires context"] }
  ]
}

POST /api/validation/batch-validate
Validate entire question-answer batch
Request: {
  "project_id": "uuid",
  "batch_number": 1
}
Response: {

```



```
"validation_report": {
  "total_q_a_pairs": 25,
  "valid": 23,
  "needs_verification": 2,
  "issues": [...]
}
```

5.7 Export Endpoints

GET /api/projects/:projectId/export?format=csv
Export project to CSV
Response: CSV file download

GET /api/projects/:projectId/export?format=json
Export project to JSON
Response: JSON file download

GET /api/projects/:projectId/export?format=markdown
Export project to Markdown table
Response: Markdown file download

GET /api/projects/:projectId/publish
Publish to production database
Request: { "backup": true, "notify_team": true }
Response: {
 "published_count": 250,
 "backup_id": "backup-uuid",
 "published_at": "2024-01-15..."
}

5.8 Audit Log Endpoints

GET /api/projects/:projectId/audit-log
Get audit trail for project
Query: ?action=edit&limit=100&offset=0
Response: {
 "data": [
 {
 "id": "log-uuid",
 "action_type": "question_edited",
 "changed_by": "user-uuid",
 "old_value": { "text": "..." },
 "new_value": { "text": "..." },
 "changed_at": "2024-01-15..."
 }
]
}

```
GET /api/audit-log/exports
Download audit report as PDF/CSV
Response: File download
```

6. KEY FEATURES TO IMPLEMENT

6.1 Table Parsing Engine

Purpose: Parse markdown tables from prompt responses into structured data

Features:

- ✓ Parse markdown table format (prompt output)
- ✓ Extract columns: Topic | Persona | Tone | Question
- ✓ Handle escaped characters and special formatting
- ✓ Validate data integrity
- ✓ Map to database schema automatically
- ✓ Handle parsing errors gracefully

Example Implementation:

```
function parseQuestionTable(markdownTable) {
  const lines = markdownTable.split('\n');
  const rows = lines
    .filter(line => line.startsWith('|'))
    .map(line => {
      const cells = line
        .split('|')
        .slice(1, -1)
        .map(cell => cell.trim());
      return {
        topic: cells[0],
        persona: cells[1],
        tone: cells[2],
        question: cells[3]
      };
    });
  return rows;
}
```

6.2 Corpus Validation Engine

Purpose: Fact-check answers against corpus file using fuzzy matching

Features:

- ✓ Extract key facts from corpus file
- ✓ Extract key facts from answers
- ✓ Fuzzy string matching (85%+ similarity threshold)
- ✓ Semantic similarity scoring
- ✓ Generate validation report with flags
- ✓ Confidence scoring

Example Implementation:

```
function validateAnswerAgainstCorpus(answer, corpus) {
  const answerKeywords = extractKeywords(answer);
  const corpusFacts = extractFacts(corpus);

  const matches = answerKeywords.map(keyword => {
    const bestMatch = corpusFacts.find(fact =>
      similarity(keyword, fact) > 0.85
    );
    return {
      keyword: keyword,
      found: !!bestMatch,
      source: bestMatch?.source
    };
  });

  const validityScore = matches.filter(m => m.found).length /
    matches.length;

  return {
    status: validityScore > 0.9 ? 'valid' : 'needs_verification',
    confidence: validityScore,
    unmatched_keywords: matches.filter(m => !m.found)
  };
}
```

6.3 Batch Processing Manager

Purpose: Handle multi-batch answer generation workflow (10 batches × 25 questions)

Features:

- ✓ Queue management (25 questions per batch)
- ✓ Sequential or parallel processing options

- ✓ Progress tracking and resumable jobs
- ✓ Error handling and retry logic
- ✓ Batch status monitoring
- ✓ Time estimates

Implementation Pseudo-code:

```
class BatchProcessor {
  async processBatch(projectId, batchNumber, questions) {
    // 1. Queue batch job
    const job = await this.createBatchJob(projectId, batchNumber);

    // 2. Call answer generator prompt
    const answers = await this.callAnswerGeneratorPrompt(
      questions,
      corpus
    );

    // 3. Parse response
    const parsedAnswers = this.parseAnswerTable(answers);

    // 4. Store in database
    await this.storeAnswers(parsedAnswers);

    // 5. Run validation
    await this.validateAnswers(parsedAnswers);

    // 6. Update job status
    await this.updateJobStatus(job.id, 'completed');
  }

  async processAllBatches(projectId) {
    const totalQuestions = await this.getQuestionCount(projectId);
    const batchCount = Math.ceil(totalQuestions / 25);

    for (let i = 1; i <= batchCount; i++) {
      const questions = await this.getQuestionsByBatch(projectId, i);
      await this.processBatch(projectId, i, questions);

      // Emit progress event
      this.emit('batch-completed', {
        batch: i,
        total: batchCount,
        progress: Math.round((i / batchCount) * 100)
      });
    }
  }
}
```

6.4 Real-time Progress Tracking

Purpose: Stream batch processing updates to UI in real-time

Features:

- ✓ WebSocket connection for real-time updates
- ✓ Progress bar updates
- ✓ Estimated time remaining calculation
- ✓ Error notifications
- ✓ Batch completion alerts

Implementation:

```
// Backend: Emit progress events
io.to(projectId).emit('batch-progress', {
  batch: 1,
  total_batches: 10,
  progress_percent: 25,
  estimated_time_remaining: 45000,
  status: 'processing'
});

// Frontend: Listen and update UI
socket.on('batch-progress', (data) => {
  updateProgressBar(data.progress_percent);
  updateEstimatedTime(data.estimated_time_remaining);
  logBatchStatus(data);
});
```

6.5 Audit & Change Tracking

Purpose: Track all edits, approvals, and publishing actions

Features:

- ✓ Log every edit with before/after values
- ✓ Track user who made change
- ✓ Timestamp all actions
- ✓ Generate audit report
- ✓ Support rollback to previous version

Implementation:

```
async function logChange(projectId, resourceType, resourceId,
  action, oldValue, newValue, userId) {
  await AuditLog.create({
    project_id: projectId,
    resource_type: resourceType, // 'question' | 'answer'
    action: action,
    old_value: oldValue,
    new_value: newValue,
    user_id: userId
  });
}
```

```

    resource_id: resourceId,
    action_type: action,           // 'created' | 'edited' | 'deleted'
    old_value: oldValue,
    new_value: newValue,
    changed_by: userId,
    changed_at: new Date()
  });
}

```

6.6 Data Migration to Production

Purpose: Safely migrate approved Q&A to production database

Features:

- ✓ Pre-migration validation checks
- ✓ Automatic backup creation
- ✓ Atomic transaction (all or nothing)
- ✓ Post-migration verification
- ✓ Rollback capability
- ✓ Change notifications

Implementation:

```

async function publishToProduction(projectId) {
  // 1. Pre-flight checks
  const validation = await validateProjectBeforePublish(projectId);
  if (!validation.success) throw new Error(validation.reason);

  // 2. Create backup
  const backup = await createDatabaseBackup();

  // 3. Start transaction
  const transaction = await db.transaction();
  try {
    // 4. Get all approved Q&A
    const content = await getApprovedContent(projectId);

    // 5. Migrate to production
    await transaction('production_qa').insert(content);

    // 6. Update project status
    await transaction('qa_projects')
      .where('id', projectId)
      .update({ status: 'published', published_at: new Date() });

    // 7. Commit
    await transaction.commit();

    // 8. Notify team
    await notifyTeam({

```

```
        type: 'publish_success',
        projectId,
        count: content.length,
        backup_id: backup.id
    });
} catch (error) {
    await transaction.rollback();
    throw error;
}
}
```

7. IMPLEMENTATION ROADMAP FOR CURSOR

7.1 Phase 1: Database & API Foundation (Week 1-2)

- ✓ Create database tables (see section 2.1)
- ✓ Set up database indexes (see section 2.2)
- ✓ Initialize Express/FastAPI server
- ✓ Implement basic CRUD endpoints for projects
- ✓ Implement corpus file upload/management endpoints
- ✓ Set up authentication middleware
- ✓ Configure database connection pooling

7.2 Phase 2: Prompt Integration (Week 2-3)

- ✓ Implement LLM API client (OpenAI/Claude/etc)
- ✓ Create table parsing engine (section 6.1)
- ✓ Implement question generator endpoint
- ✓ Implement answer generator endpoint
- ✓ Add prompt versioning system
- ✓ Error handling for failed API calls

7.3 Phase 3: Validation Engine (Week 3-4)

- ✓ Implement corpus validation engine (section 6.2)
- ✓ Create fuzzy matching algorithm
- ✓ Implement batch validation endpoint
- ✓ Build validation reporting
- ✓ Add confidence scoring

7.4 Phase 4: Frontend Dashboard (Week 4-6)

- ✓ Create React/Vue component library
- ✓ Build main dashboard screen (section 4.1)
- ✓ Build project wizard (section 4.2)
- ✓ Build question editor grid (section 4.3)
- ✓ Build answer variant selector (section 4.4)
- ✓ Implement real-time data refresh

7.5 Phase 5: Batch Processing & Workflows (Week 6-7)

- ✓ Implement batch processor (section 6.3)
- ✓ Build batch processing UI (section 4.5)
- ✓ Implement WebSocket real-time updates (section 6.4)
- ✓ Add progress tracking
- ✓ Implement error recovery

7.6 Phase 6: Approval & Publishing (Week 7-8)

- ✓ Build approval workflow UI (section 4.6)
- ✓ Implement audit logging (section 6.5)
- ✓ Build publishing engine (section 6.6)

- ✓ Implement backup/restore
- ✓ Add export options (CSV, JSON, Markdown)

7.7 Phase 7: Testing & Documentation (Week 8-9)

- ✓ Unit tests for all services
- ✓ Integration tests for workflows
- ✓ Load testing for batch processing
- ✓ User acceptance testing
- ✓ API documentation (Swagger/OpenAPI)

7.8 Phase 8: Deployment & Optimization (Week 9-10)

- ✓ Deploy to staging environment
- ✓ Performance optimization
- ✓ Security hardening
- ✓ Monitor error rates and fix issues
- ✓ Production deployment

8. TECHNOLOGY STACK RECOMMENDATIONS

Backend:

- Node.js + Express
- OR Python + FastAPI
- For scalability + performance

Database:

- PostgreSQL (primary)
- Redis (caching/sessions)
- S3/MinIO (file storage)

Frontend:

- React 18+ with TypeScript
- OR Vue 3 with TypeScript
- TailwindCSS for styling

Real-time:

- Socket.IO for WebSockets
- Message queues: Bull/RabbitMQ
- For batch processing

Data Tables:

- AG-Grid (advanced features)
- React Table (lightweight)
- For question/answer editing

LLM Integration:

- OpenAI API client
- OR Anthropic Claude API
- Langchain for prompt management

9. CRITICAL IMPLEMENTATION NOTES FOR CURSOR

Key Constraints

- **Table Format Consistency:** Question Generator output MUST parse exactly as Answer Generator input. Use strict regex validation.
- **Context Window Management:** Answer Generator processes max 25 questions per call. Enforce batch limits in API.
- **Corpus File Size:** Keep corpus file under 50-100 KB. Implement compression if needed.
- **API Rate Limiting:** LLM API calls are expensive. Implement caching and avoid re-calling with same input.
- **Database Transaction Integrity:** Publishing must be atomic. No partial migrations.

⚠ Error Handling & Edge Cases

- Handle malformed markdown tables from prompt responses
- Gracefully handle LLM API timeouts/failures (retry logic)
- Handle corpus file encoding issues (UTF-8, special characters)
- Prevent duplicate questions if user regenerates

- Validate all user inputs before sending to LLM
- Handle browser disconnects during long batch processing

Performance Optimization Tips

- Use database connection pooling (min 5, max 20 connections)
- Implement Redis caching for frequently accessed corpuses
- Use pagination for all list endpoints (max 100 per page)
- Lazy-load answer variants only when needed
- Implement batch inserts (INSERT with multiple rows) for speed
- Use indexes on project_id, status, batch_number columns

10. CONFIGURATION & ENVIRONMENT VARIABLES

```
# Database
DATABASE_URL=postgresql://user:pass@localhost:5432/qa_studio
REDIS_URL=redis://localhost:6379
DATABASE_POOL_MIN=5
DATABASE_POOL_MAX=20

# LLM API
LLM_API_KEY=sk-...
```

```
LLM_API_BASE=https://api.openai.com/v1
LLM_MODEL=gpt-4
LLM_MAX_TOKENS=8000
LLM_TIMEOUT_MS=60000

# Application
PORT=3000
NODE_ENV=production
CORS_ORIGIN=https://yourdomain.com
SESSION_SECRET=random-secret-key
MAX_FILE_SIZE_MB=10

# Batch Processing
BATCH_SIZE=25
MAX_CONCURRENT_BATCHES=3
BATCH_TIMEOUT_MS=300000

# Logging
LOG_LEVEL=info
LOG_FILE=/var/log/qa-studio.log

# Backups
BACKUP_ENABLED=true
BACKUP_SCHEDULE=0 2 * * * # 2 AM daily
BACKUP_RETENTION_DAYS=30
BACKUP_LOCATION=/backups

# Email Notifications
SMTP_HOST=smtp.gmail.com
SMTP_PORT=587
SMTP_USER=noreply@company.com
SMTP_PASS=app-password
NOTIFY_ON_PUBLISH=true
```

11. SECURITY CONSIDERATIONS

- ✓ Implement role-based access control (RBAC)
- ✓ Use JWT tokens for API authentication
- ✓ Validate all file uploads (size, type, content)
- ✓ SQL injection prevention (use parameterized queries)
- ✓ XSS prevention (sanitize user inputs)
- ✓ Rate limiting on API endpoints

- ✓ Audit logging for all sensitive operations
- ✓ Encrypt sensitive data (API keys, passwords)
- ✓ HTTPS-only communications
- ✓ Regular security audits and penetration testing

12. TESTING STRATEGY

Unit Tests:

- Table parsing engine (100+ test cases)
- Corpus validation logic
- Batch processor logic
- All utility functions

Integration Tests:

- Full question generation workflow
- Full answer generation workflow
- Publishing workflow
- Database transactions

Load Tests:

- Process 1,000+ questions per minute
- Handle 100+ concurrent users
- Batch processing stability

User Acceptance Tests:

- Complete E2E workflows
- UI responsiveness
- Error handling scenarios
- Data integrity after publishing

Recommended Testing Stack:

- Jest (unit tests)
- Supertest (API tests)
- Cypress (E2E tests)
- k6 or JMeter (load tests)

13. DEPLOYMENT CHECKLIST

- ☐ Database migrations completed
- ☐ All environment variables configured
- ☐ SSL certificates installed
- ☐ Backup system tested
- ☐ Monitoring and alerting set up
- ☐ Logging centralized (ELK stack or similar)
- ☐ Rate limiting configured
- ☐ CORS properly configured
- ☐ API documentation deployed
- ☐ User training completed
- ☐ Rollback procedure documented
- ☐ On-call support established

14. SUPPORT & MAINTENANCE

- Monitor LLM API costs and usage
- Regular database maintenance (VACUUM, ANALYZE)
- Clean up old batch jobs and logs
- Update prompt versions as needed
- Patch security vulnerabilities promptly
- Implement feature requests from content team
- Maintain API documentation
- Support team for content professionals

CONCLUSION

This development guide provides Cursor with all specifications needed to build a production-ready Q&A Content Studio. The system integrates your existing database with the dual-prompt workflow (Question Generator + Answer Generator), enabling content professionals to manage the complete lifecycle of AI-generated Q&A content through an intuitive GUI.

Key deliverables:

- Full database schema with 6 core tables
- 25+ RESTful API endpoints
- Complete UI component specifications
- 6 critical business logic engines
- 10-week implementation roadmap
- Security, testing, and deployment guidance

Start with Phase 1 (database + basic API), then proceed sequentially through Phase 8. Each phase builds on previous phases and can be tested independently.